Lab 9: Implement and Analyze Prim's MST Algorithm

Theory:

Prim's algorithm is a greedy algorithm that finds a Minimum Spanning Tree (MST) for a weighted undirected graph. It starts with an empty spanning tree and maintains two sets of vertices: vertices included in the MST and vertices not yet included. At every step, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges.

The primary difference from Kruskal's algorithm is that Prim's algorithm maintains a single continuous tree throughout its execution, whereas Kruskal's builds a forest and merges the trees together. Prim's is often implemented using an adjacency matrix for dense graphs or an adjacency list with a priority queue for sparse graphs.

Space Complexity: O(V)

Expressing Complexity of cases

Case	Time Complexity	Condition
Average	$O(V^2)$ or $O(E + V \log V)$	Adj. Matrix or Fib. Heap
Worst	$O(V^2)$ or $O(E + V \log V)$	Dense graph

Algorithm

1. Initialize a tree with a single vertex, chosen arbitrarily from the graph.
2. Grow the tree by one edge: of the edges that connect the tree to vertices not yet in the tree, find the minimum-weight edge, and transfer it to the tree.
3. Repeat step 2 until all vertices are in the tree.

Pseudocode

```
BEGIN
  FOR each vertex u in V DO
    key[u] ← ∞
    parent[u] ← NIL
  END FOR
  key[root] ← 0
  Q ← V
  WHILE Q is not empty DO
    u ← EXTRACT-MIN(Q)
    FOR each vertex v in Adj[u] DO
      IF v □ Q and weight(u, v) < key[v] THEN
        parent[v] ← u
        key[v] ← weight(u, v)
      END IF
    END FOR
  END WHILE
END
```

Conclusion:

The experiment successfully demonstrates Prim's algorithm navigating a weighted, undirected adjacency matrix to identify the Minimum Spanning Tree. Beginning from an initial source vertex, the algorithm iteratively expanded the MST by picking the minimum-weight edge connecting the tree vertices to the non-tree vertices. The resulting MST connects vertices A→B (Weight: 2), A→D (Weight: 6), B→C (Weight: 3), and B→E (Weight: 5), yielding a minimum total cost of 16.

Unlike Kruskal's algorithm, which can form a forest during intermediate steps, Prim's guarantees that the intermediate graph is always a single connected tree. Using a simple adjacency matrix approach, this method elegantly ran in $O(V^2)$ time. Prim's algorithm proves highly efficient for dense networks where the number of edges is high, and is commonly used in network design and protocol routing pathways.